AI-Based Financial Time Series Forecasting using LSTM

This project focuses on predicting the **next day closing price** of Reliance stock using **Deep Learning (LSTM – Long Short-Term Memory)**.

Stock market data is **time series data**, where:

- Each value depends on previous values
- Patterns exist over time
- Sequential learning is required

Traditional ML fails here
LSTM handles sequence learning effectively

Dataset Description

The dataset contains historical stock data of Reliance.

Columns:

Column	Description
Date	Trading date
Open	Price at market opening
High	Highest price of the day
Low	Lowest price of the day
Close	Final price of the day
Volume	Total shares traded

1. Dataset Columns

The dataset contains stock market information of Reliance company.

Date
It represents the trading date.

Open
The price of the stock when the market opened on that day.

High
The highest price reached by the stock on that day.

Low
The lowest price reached by the stock on that day.

Close
The final price of the stock when the market closed on that day.

Volume
The total number of shares traded on that day.

Example:

```
Date        Open     High     Low      Close    Volume
2024-01-01  2505.4   2520.8   2490.2   2510.5   5000000
```

Here, = 2510.5 means Reliance stock ended that trading day at ₹2510.5.

Close Price :

In this project, we select only the **Close** price.

```
close_data = data[["Close"]].values
```

The close price is very important because it represents the final agreed value of the stock for that trading day.

```
Output : Future closing price
```

Example:

```
Day 1 Close = 2500
Day 2 Close = 2515
Day 3 Close = 2530
```

The model observes previous close prices and tries to predict the next close price.

Data Preprocessing

Step 1: Sorting Data

```
data = data.sort_values("Date")
```

Ensures chronological order

Step 2: Scaling Data

We use:

```
MinMaxScaler(feature_range=(0,1))
```


Formula:

```
Scaled = (Value - Min) / (Max - Min)
```

Example:

```
Min = 2500
Max = 2700
Value = 2600

Scaled = (2600 - 2500) / (200)
       = 0.5
```

Neural networks perform better with small values

Page 3 / 14

Time Series Sequence Generation

LSTM does not take only one value. It learns from a sequence of previous values.

In our code:

```
TIME_STEPS = 10
```

This means:

```
Previous 10 days closing prices are used to predict the 11th day
closing price.
```

Example:

```
Input X:
Day 1  = 2500
Day 2  = 2510
Day 3  = 2525
Day 4  = 2530
Day 5  = 2540
Day 6  = 2555
Day 7  = 2560
Day 8  = 2570
Day 9  = 2585
Day 10 = 2590
```

```
Day 11 = 2600
```

Next sequence:

```
Input X:
Day 2 to Day 11

Output y:
Day 12
```

This is called **sliding window technique**.

LSTM Input Shape

LSTM expects input in 3D format.

```
(samples, time_steps, features)
```

In our project:

```
X = X.reshape(X.shape[0], X.shape[1], 1)
```

Meaning:

```
samples    = Total number of training examples
time_steps = 10 previous days
features   = 1 because we use only Close price
```

Example:

```
X shape = (100, 10, 1)
```

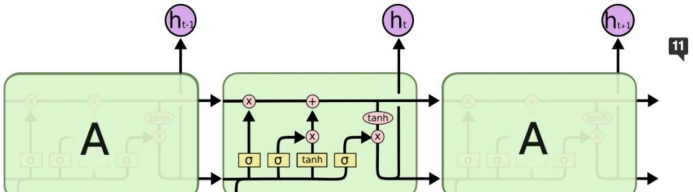
Meaning:

```
100 samples
Each sample contains 10 days
Each day contains 1 value: Close price
```


Understanding LSTM

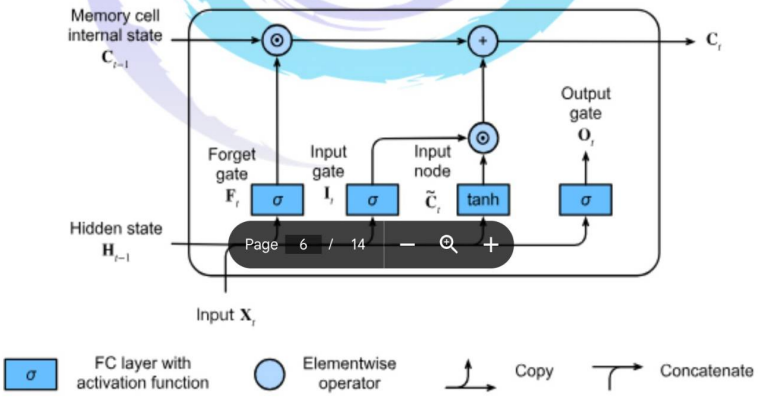
LSTM is a special type of RNN designed to:

Remember long-term dependencies
Avoid vanishing gradient problem



The repeating module in an LSTM contains four interacting layers.

LSTM Components



LSTM means **Long Short-Term Memory**.

It is used to remember important past information and forget unnecessary information.

LSTM has mainly 3 gates:

Forget Gate

Decides what old information should be forgotten.

Example:

```
Old market trend is no longer useful, so forget it.
```

Input Gate

Decides what new information should be added.

Example:

```
Today's price movement is important, so store it.
```

Output Gate

Decides what information should be sent as output.

Example:

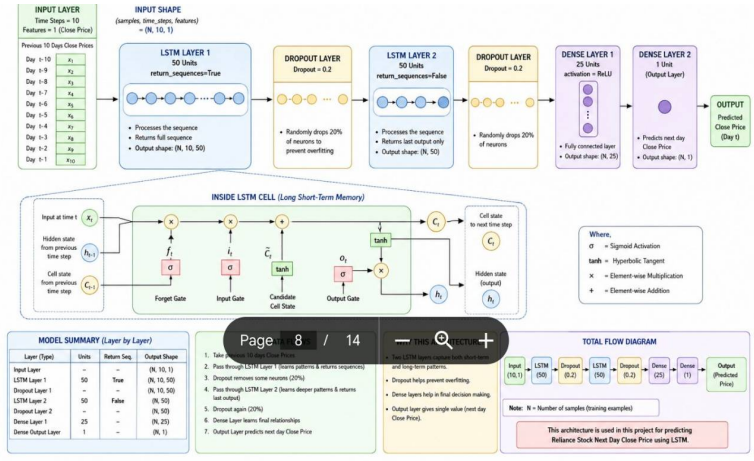
```
Use important memory to predict tomorrow's close price.
```

Simple explanation:

```
Forget Gate  → Removes useless memory
Input Gate   → Adds new useful memory
Output Gate  → Produces prediction
```

That is why LSTM is better than simple RNN for time series data.

Model Architecture



Our model architecture is:

```
model.add(LSTM(units=50, return_sequences=True,
input_shape=(TIME_STEPS, 1)))
model.add(Dropout(0.2))

model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))

model.add(Dense(units=25, activation="relu"))
model.add(Dense(units=1))
```


Explanation:

First LSTM Layer

```
Learns sequence pattern from previous 10 days.
```

return_sequences=True

```
It sends output sequence to the next LSTM layer.
```

Dropout Layer

```
Randomly ignores 20% neurons during training.
It helps to reduce overfitting.
```

Second LSTM Layer

```
Learns deeper time-based patterns.
```

Dense Layer with 25 neurons

```
Works as fully connected layer for final decision-making.
```

Final Dense Layer with 1 neuron

```
Gives one output: predicted close price.
```

Layer-wise Explanation:

LSTM Layer 1

- Learns sequence patterns
- Returns full sequence

Dropout

- Prevents overfitting
- Randomly removes neurons

LSTM Layer 2

- Learns deeper patterns
- Returns final output only

Dense Layer (25)

- Learns complex relationships

Output Layer

- Predicts single value (next day price)

Model Training

```
model.fit(X_train, y_train)
```

Key Parameters:

Parameter	Meaning
Epochs	Number of iterations
Batch Size	Data processed at once
Validation Split	Data for testing during training

Loss Function:


```
loss = mean_squared_error
```

Meaning:

```
Difference between actual and predicted value
```

Lower loss = better model

Training and Loss

Training means model learns from past data.

```
model.fit(
    X_train,
    y_train,
    epochs=60,
    batch_size=16,
    validation_split=0.2
)
```


Epoch

One complete pass over the training data.

```
epochs = 60
```

Means model can learn from data up to 60 times.

Batch Size

```
batch_size = 16
```

Means model processes 16 samples at a time.

Loss

Loss means error.

```
Loss = Difference between actual value and predicted value
```

In this project, we use:

```
loss="mean_squared_error"
```

If loss decreases, model is learning properly

Example:

```
Epoch 1  Loss = 0.050
Epoch 10 Loss = 0.020
Epoch 30 Loss = 0.006
```

This shows improvement.

Predictions

After training, we test the model:

```
predicted_scaled = model.predict(X_test)
```

Model gives predicted values in scaled format.

Example:

```
Predicted scaled value = 0.65
```

But this is not original stock price.

So we need inverse scaling to convert it back into rupees.

Inverse Scaling

During preprocessing, we converted prices into values between 0 and 1.

Now after prediction, we convert scaled values back to original prices.

```
predicted_prices = scaler.inverse_transform(predicted_scaled)
actual_prices = scaler.inverse_transform(y_test.reshape(-1, 1))
```

Manual formula:

```
Original Value = Scaled Value × (Maximum Value - Minimum Value) +
Minimum Value
```

Example:

```
Minimum Price = 2500
Maximum Price = 2700
Scaled Prediction = 0.75
```

Calculation:

```
Original Price = 0.75 × (2700 - 2500) + 2500
Original Price = 0.75 × 200 + 2500
Original Price = 150 + 2500
Original Price = 2650
```

So, predicted stock price is ₹2650.

MAE, MSE, RMSE

These are evaluation metrics.

They tell us how much mistake the model is making.

MAE: Mean Absolute Error

Formula:

```
MAE = Average of absolute errors
```

Example:

```
Actual    = 2600
Predicted = 2580

Error = |2600 - 2580| = 20
```

MAE is easy to understand because it is in the same unit as price.

If:

```
MAE = 25
```

It means average prediction error is around ₹25.

MSE: Mean Squared Error

Formula:

```
MSE = Average of squared errors
```

Example:

```
Error = 20
Squared Error = 20 × 20 = 400
```

MSE gives more penalty to large errors.

RMSE: Root Mean Squared Error

Formula:

```
RMSE = Square root of MSE
```


Example:

```
MSE = 400
RMSE = √400 = 20
```

RMSE is also in the same unit as stock price.

In code:

```
mse = mean_squared_error(actual_prices, predicted_prices)
mae = mean_absolute_error(actual_prices, predicted_prices)
rmse = np.sqrt(mse)
```

Next-Day Prediction

For next-day prediction:

```
last_10_days = scaled_close[-TIME_STEPS:]
last_10_days = last_10_days.reshape(1, TIME_STEPS, 1)

next_day_scaled = model.predict(last_10_days)
next_day_price = scaler.inverse_transform(next_day_scaled)
```

Meaning:

```
Take last 10 days closing prices
Give them to LSTM model
Model predicts next day close price
Convert prediction back to original price
```

Example:

```
Last 10 days close prices → Input
Tomorrow close price      → Output
```

Final output:

```
Predicted Next Day Close Price: ₹2635.75
```

This is the final forecast of the model.

Project Summary (That you can tell in interview)

"I developed an LSTM-based time series forecasting model to predict stock prices using historical data. I performed preprocessing, sequence generation, model training, evaluation, and future prediction."


```
Error = |2600 - 2580| = 20
```

MAE is easy to understand because it is in the same unit as price.

If:

```
MAE = 25
```